Torque and OJB

The Apache persistence frameworks

Martin Pöschl
mpoeschl@marmot.at

Agenda

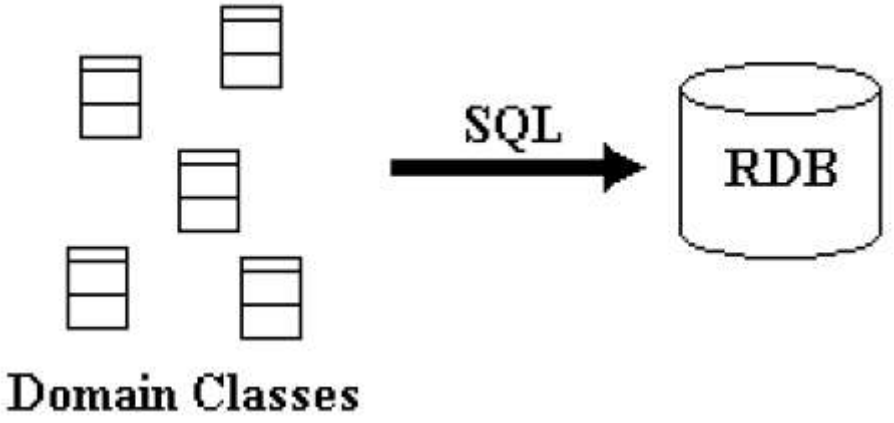
- Introduction persistence frameworks
- Torque
  - Torque-generator
- ObjectRelationalBridge (OJB)
- Torque or OJB?

Bio: Martin Pöschl

ANECON

- Senior Developer at ANECON
  - Developing web-solutions
- Apache commiter since 2000
- involved in the development of
  - Torque
  - OJB
  - Turbine
  - ...
- Member of the Jakarta PMC

Persistence without framework



- **Hard-coding SQL in your domain/business classes.**
  - **SQL embedded in Java source code**
- **Allows you to write code very quickly.**
- **Directly couples your business classes with the schema of your relational database.**
- **Results in code that is difficult to maintain and extend.**

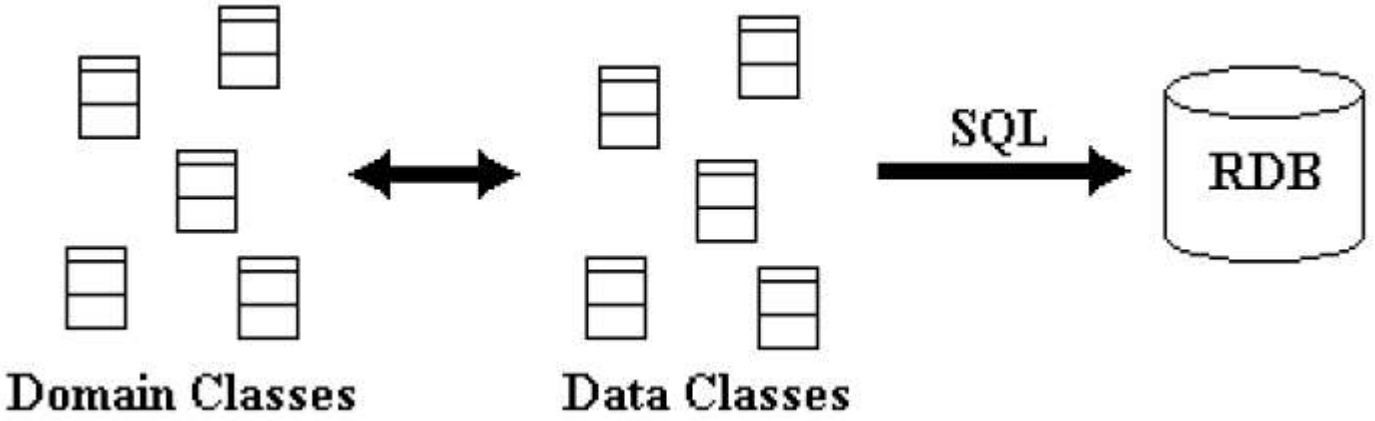
Hardcoded SQL

```
String url = "jdbc:mySubprotocol:myDataSource";
String query = "select COF_NAME, PRICE from COFFEES";

try {
    Class.forName("myDriver.ClassName");
} catch(Exception ex) {
    setError("Can't find Database driver class: " + ex);
    return;
}

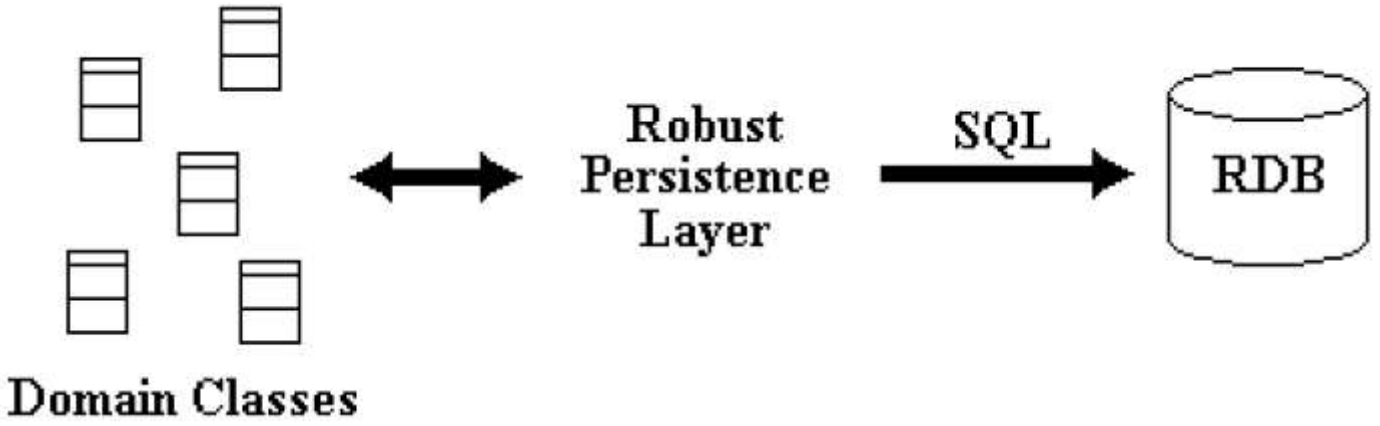
try {
    Vector results = new Vector();
    Connection con = DriverManager.getConnection(url, "myLogin", "myPassword");
    Statement stmt = con.createStatement();
    ResultSet rs = stmt.executeQuery(query);
    while (rs.next()) {
        String s = rs.getString("COF_NAME");
        float f = rs.getFloat("PRICE");
        String text = s + "     " + f;
        results.addElement(text);
    }
    stmt.close();
    con.close();
    setResults(results);
} catch(SQLException ex) {
    setError("SQLException: " + ex);
}
```

Persistence frameworks



- **Creating data classes corresponding to domain/business classes.**
- **Encapsulates the source code that handles the hard-coded interactions**
- **Changes to the schema (RDB) affect your code.**
- **Code generation reduces the handwritten lines of code**

Persistence frameworks



- Persistence layer maps objects to relational databases.
- Simple changes to the schema (RDB) do not affect your code.
- Your application programmers do not need to know a thing about the schema of the relational database.
- Disadvantage: there is a performance impact to your applications

ANECON

Torque
http://db.apache.org/torque/

Torque

ANECON

- Developed as part of Turbine
- Torque 3.0 was the first 'stand-alone' version
- Torque 3.1 is the current release
- Uses generated Peer classes
- Manager classes to handle caching (JCS)
- Supports all major RDBMS
- id generation (IDBroker or native)
- commons-dbcp connection pool
- Criteria class to build 'where clause'
  - You don't have to write SQL

Torque OM

ANECON

- DB Table: APACHE_COMMITER
- Torque OM
  - BaseApacheCommiter
    - Contains all getter and setter methods
    - Generated by Torque
  - ApacheCommiter
    - Used by the developer to add additional methods
    - only generated if it does not exist
  - BaseApacheCommiterPeer
    - Contains basic methods to load and store objects
    - Generated by Torque
  - ApacheCommiterPeer
    - Used by the developer to add additional methods
    - only generated if it does not exist

Torque API

ANECON

- Example: (ApacheCommiterPeer)

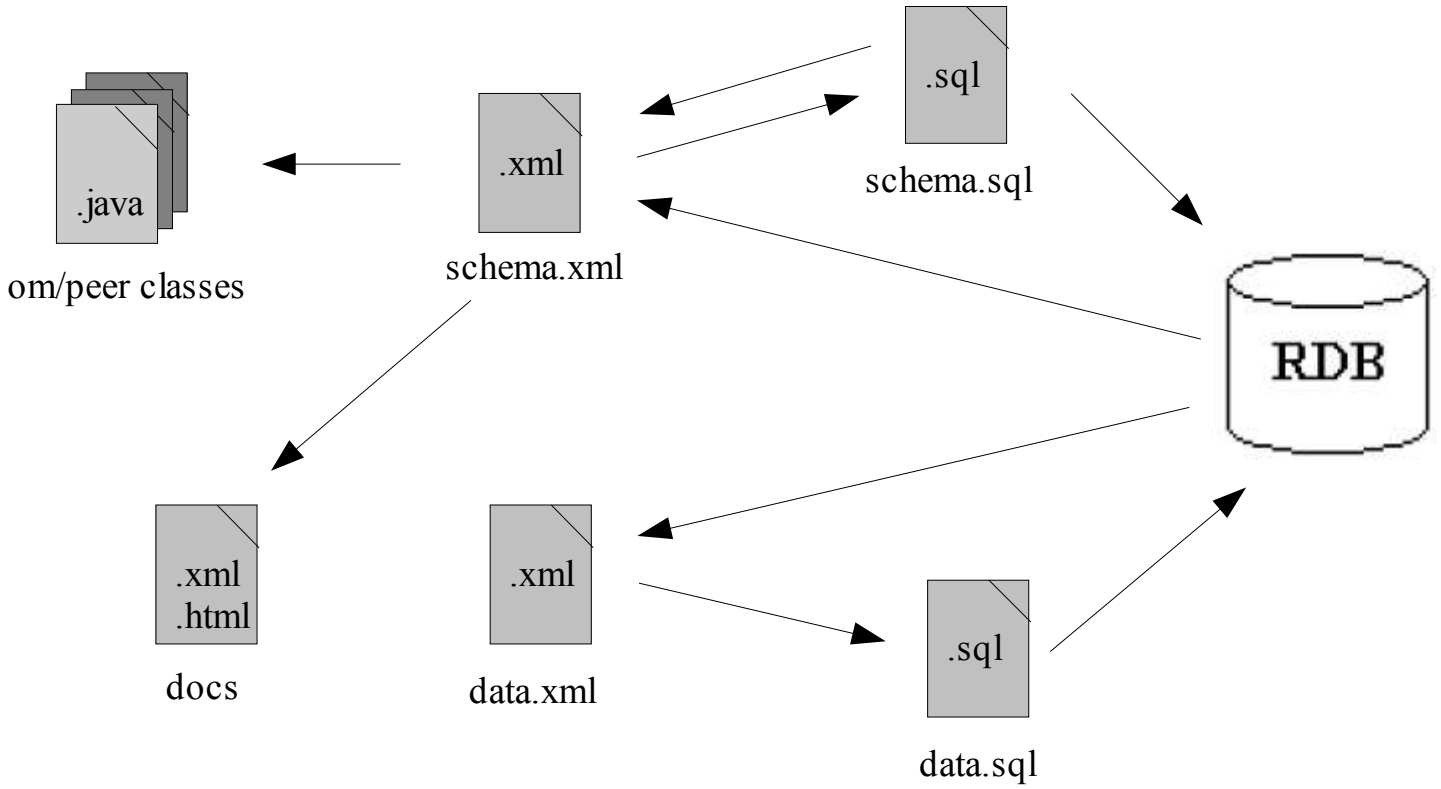
```
public static List getCommitersByCountry(String countyCode) {
    List result = null;
    try {
        Criteria crit = new Criteria();
        crit.add(ApacheCommiterPeer.COUNTRY_CODE, countyCode);
        result = ApacheCommiterPeer.doSelect(crit);
    } catch (Exception ex) {
        log.error(ex, ex);
    }
    return result;
}
```

Torque-generator

ANECON

- Generates the om/peer classes for Torque
- Generates SQL for your target RDB
  - Modell SQL
  - Data SQL
- Generates html or xml docs
- Supports Ant (Distribution includes build-torque.xml)
- Maven-plugin (Part of Maven)
- Reverse engineering from existing database
  - schema.xml
  - data.xml
- Support for OJB
  - Simple object-modell
  - repository.xml

Torque-generator



Torque-generator

ANECON

- Sample schema.xml

```
<?xml version="1.0" encoding="ISO-8859-1" standalone="no"?>
<!DOCTYPE database SYSTEM "http://db.apache.org/torque/dtd/database_3_1.dtd">

<database name="TURBINE">
  <table name="TURBINE_USER" idMethod="idbroker">
    <column name="USER_ID" required="true" primaryKey="true" type="INTEGER"/>
    <column name="LOGIN_NAME" required="true" size="64" type="VARCHAR"
            javaName="UserName"/>
    <column name="PASSWORD_VALUE" required="true" size="16" type="VARCHAR"
            javaName="Password"/>
    <column name="FIRST_NAME" required="true" size="64" type="VARCHAR"/>
    <column name="LAST_NAME" required="true" size="64" type="VARCHAR"/>
    <column name="LAST_LOGIN" type="TIMESTAMP"/>
    <unique>
      <unique-column name="LOGIN_NAME"/>
    </unique>
  </table>
  <table name="TURBINE_ROLE_PERMISSION">
    <column name="ROLE_ID" required="true" primaryKey="true" type="INTEGER"/>
    <column name="PERMISSION_ID" required="true" primaryKey="true" type="INTEGER"/>
    <foreign-key foreignTable="TURBINE_ROLE">
      <reference local="ROLE_ID" foreign="ROLE_ID"/>
    </foreign-key>
    <foreign-key foreignTable="TURBINE_PERMISSION">
      <reference local="PERMISSION_ID" foreign="PERMISSION_ID"/>
    </foreign-key>
  </table>
</database>
```

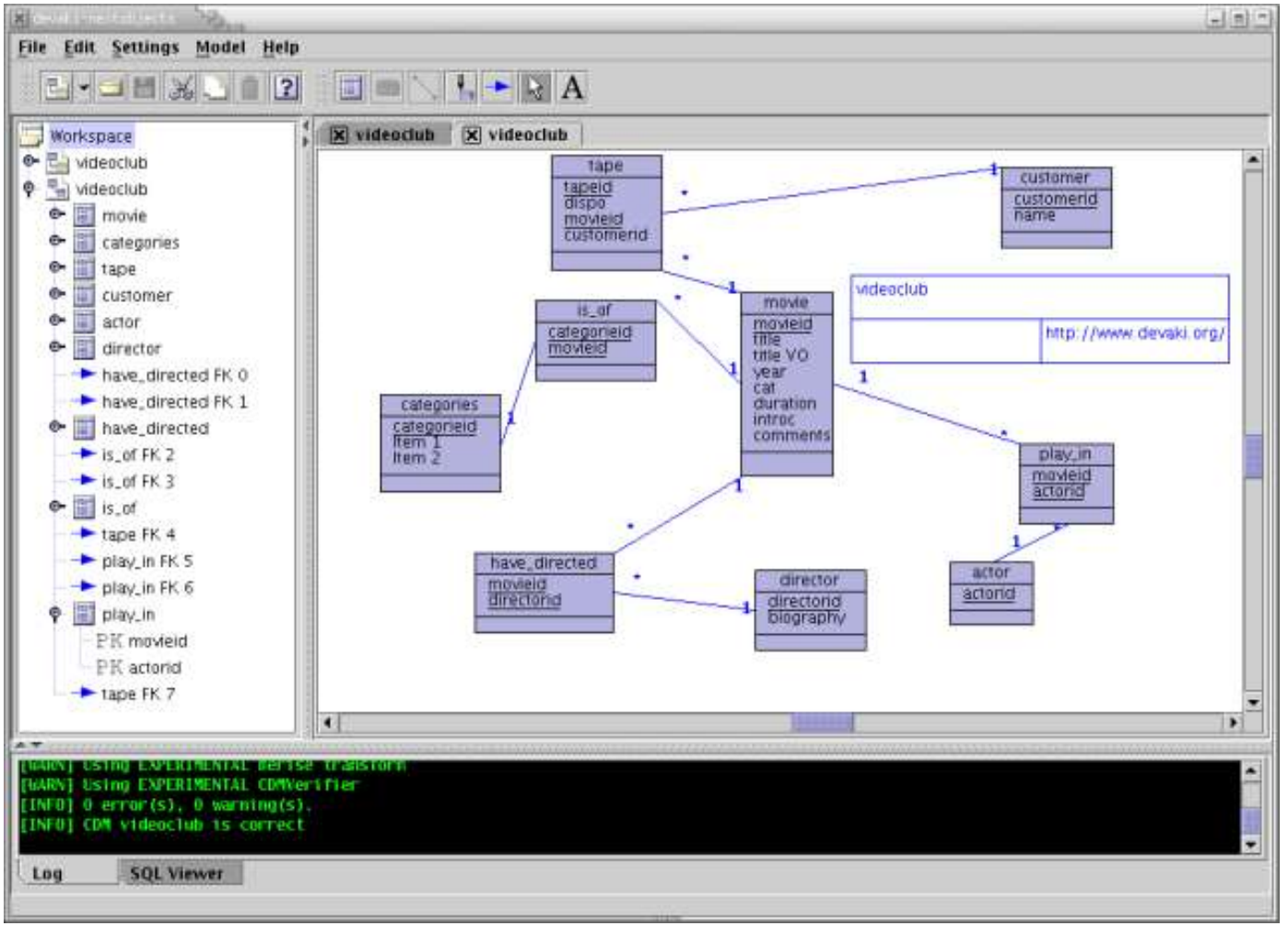
Torque-generator

ANECON

- Uses Velocity as template-engine
  - Write your own templates (e.g. to generate a simple web- or Swing-GUI)
- Used by OJB to generate the database for unit tests
- Used by the Turbine security service

Torque-generator GUI

- http://www.devaki.org



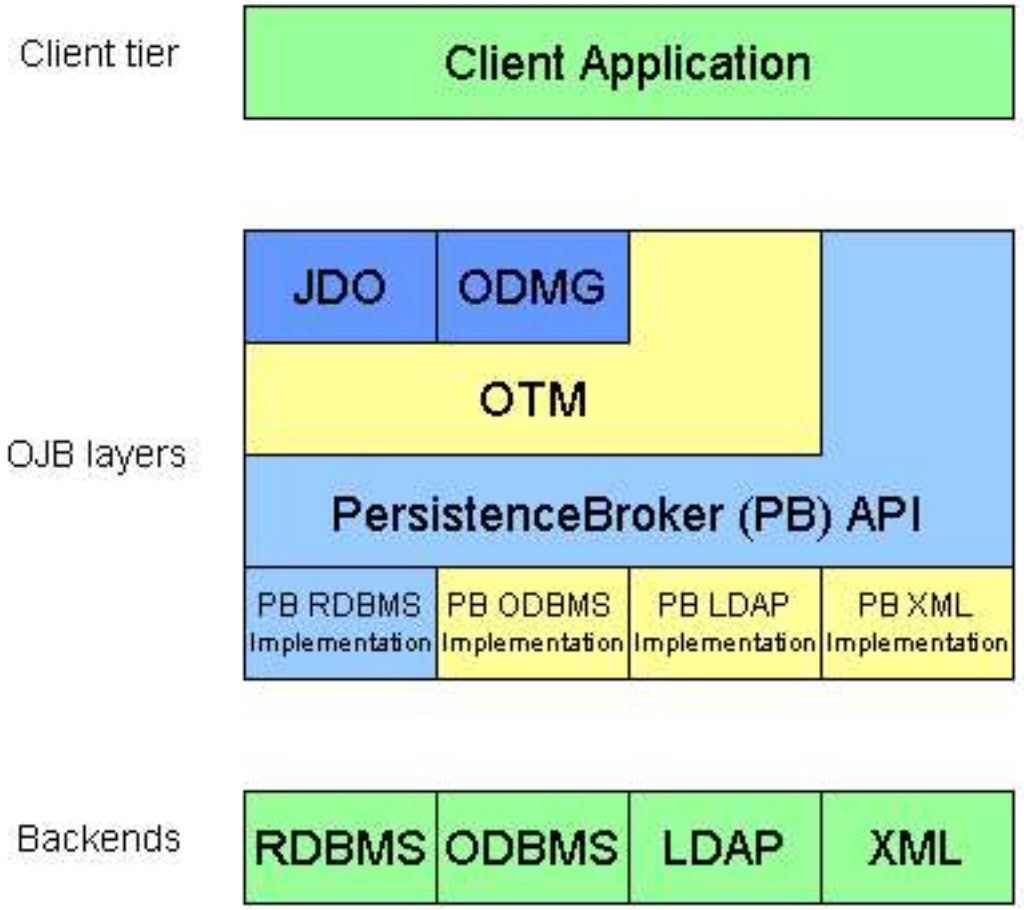
Object Relational Bridge

OJB – Object Relational Bridge

ANECON

- Originally release in Oct. 2000 (sourceforge)
- Now part of the Apache DB Project
- Allows transparent persistence
- Mappings are defined in an XML Repository
- Support for Multiple APIs
  - PersistenceBroker API
  - A **ODMG 3.0** compliant API
  - A Plugin for **JDO**

OJB



Layers in yellow are not implemented yet.

OJB features

ANECON

- Mapping support for 1:1, 1:n and m:n
- Configurable collection queries to control loading of relationships
- Lazy Materialization
- Caching
- Sequence-Managing
- prefetched relationships
- Pessimistic and Optimistic Locking
- Cache synchronization for distributed caches

Persistence Broker API

- provides a minimal API
- Similar to Torque API

Example: (ApacheCommiterManager)

```
protected Produkt getCommitersByCountry(String countyCode) {
    PersistenceBroker broker = getBroker();
    Criteria crit = new Criteria();
    crit.addEqualTo("countyCode", countyCode);
    Query query = new QueryByCriteria(ApacheCommiter.class, crit);
    List result = (List) broker.getCollectionByQuery(query);
    broker.close();
    return result;
}
```

JDO support

ANECON

- Java Data Objects (JDO) reference implementation released April 2002
- OJB has no JDO implementation
- OJB provides a plugin to the JDO reference implementation

Torque or OJB?

ANECON

- Both frameworks
  - Support for all major RDBMS
    - MySQL
    - Oracle
    - DB2
    - ...
  - Connection pooling
  - Object Cache
  - Generation of unique ids
  - 100 %: pure Java, Open Source, Apache License

Torque or OJB?

ANECON

- Torque advantages
  - Easy to use (base system is generated)
  - Simple API
- OJB advantages
  - Better designed
  - ODMG and JDO support
  - integrates smoothly into J2EE Application servers

Torque or OJB?

ANECON

- Depends on your needs
- Read the docs
- Browse the mailing-list archives
- Look at the unit tests
- Ask questions on the user lists
- Ask now ;-)

References

- Scott Ambler
  - http://www.ambysoft.com
- Object Data Management Group (ODMG)
  - http://www.odmg.org/
- Java Data Objects (JDO)
  - http://java.sun.com/products/jdo/
- Updated slides + Code samples
  - http://www.apache.org/~mpoeschl/ac2003-mo15/

This presenation has ended.
Click anywhere to return.

_

